ASSIGNMENT - 1

HTML, CSS & Bootstrap – Complete Revision Handbook

Scope: HTML5 + CSS3 + Bootstrap 5
Level: Beginner to Intermediate
Purpose: Understanding, practical development, interviews and quick revision.

MODULE 1 – HTML FUNDAMENTALS

1. What is HTML?

HTML stands for HyperText Markup Language.

- Creates webpage structure.
- Defines headings, paragraphs, images, links, tables and forms.
- Uses elements represented by tags.
- HTML is a markup language, not a programming language.

2. Basic HTML structure

```
None
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>My Website</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>My first webpage.</p>
</body>
</html>
```

Element	Purpose

`<!DOCTYPE html>`	Declares HTML5
`<html>`	Root element
`<head>`	Metadata
`<title>`	Browser tab title
`<meta>`	Metadata and viewport settings
`<body>`	Visible webpage content

3. HTML tags and elements

```
None
<p>Hello World</p>
```

- `<p>` – Opening tag.
- `Hello World` – Content.
- `</p>` – Closing tag.

Paired elements:

```
None
<h1>Welcome</h1>
<p>Hello</p>
```

Void elements:

```
None
<br>
<hr>
<img src="photo.jpg" alt="Photo">
<input type="text">
```

Void elements do not require closing tags.

4. Headings

HTML provides six heading levels.

```
None
<h1>Main Heading</h1>
<h2>Subheading</h2>
<h3>Section Heading</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

`h1` is the highest-level heading and `h6` is the lowest.

5. Paragraphs and text formatting

```
None
<p>Normal paragraph</p>
<b>Bold text</b>
<strong>Important text</strong>
<i>Italic text</i>
<em>Emphasized text</em>
<u>Underlined text</u>
<mark>Highlighted text</mark>
<del>Deleted text</del>
<small>Small text</small>
```

Important difference:

- `b` – Visual bold formatting.
- `strong` – Strong importance.
- `i` – Alternate voice or style.
- `em` – Emphasis.

6. HTML lists

Ordered list:

```
None
<ol>
    <li>HTML</li>
    <li>CSS</li>
    <li>Bootstrap</li>
</ol>
```

Unordered list:

```
None
<ul>
    <li>Car</li>
    <li>Bike</li>
    <li>Bus</li>
</ul>
```

Description list:

```
None
<dl>
    <dt>HTML</dt>
    <dd>Structures webpages.</dd>
</dl>
```

7. Hyperlinks

```
None
<a href="https://www.google.com">
    Visit Google
</a>
```

Open in another tab:

```
None
<a href="https://www.google.com"
   target="_blank"
   rel="noopener noreferrer">
    Google
</a>
```

- `href` – Destination.
- `target` – Where to open the link.
- `rel` – Relationship and security behavior.

8. Images

```
None
<img src="car.jpg"
     alt="Red car"
     width="300">
```

Attribute	Purpose
`src`	Image location
`alt`	Alternative text
`width`	Image width
`height`	Image height

9. HTML attributes

Attributes provide additional information about elements.

```
None
<input type="text"
       id="username"
       class="form-control"
       placeholder="Enter name"
       required>
```

Important attributes:

- `id` – Unique identifier.
- `class` – Reusable styling identifier.
- `type` – Element/input type.
- `placeholder` – Input hint.
- `required` – Mandatory field.
- `value` – Default value.
- `disabled` – Disables the control.

10. ID vs Class

```
None
<h1 id="title">Welcome</h1>

<p class="highlight">First paragraph</p>
```

```
<p class="highlight">Second paragraph</p>
```

```
None
#title {
    color: blue;
}

.highlight {
    color: red;
}
```

ID	Class
Intended to be unique	Can be reused
CSS selector `#`	CSS selector `.`
Identifies an element	Groups elements

MODULE 2 – HTML FORMS, TABLES AND SEMANTIC ELEMENTS

1. HTML Forms

Forms collect user information.

```
None
<form>
    <label for="name">Name:</label>

    <input type="text"
           id="name"
           name="name"
           required>

    <button type="submit">Register</button>
```

```
</form>
```

2. Different input types

```
None
<input type="text">
<input type="password">
<input type="email">
<input type="number">
<input type="date">
<input type="tel">
<input type="radio">
<input type="checkbox">
<input type="file">
```

Important attributes:

Attribute	Purpose
`required`	Mandatory
`min`	Minimum value
`max`	Maximum value
`minlength`	Minimum text length
`maxlength`	Maximum text length
`readonly`	Cannot be edited
`disabled`	Disabled control
`checked`	Initially selected
`placeholder`	Input hint

3. Radio buttons vs Checkboxes

Radio buttons: Select one option from a group.

```
None
<input type="radio" name="gender" value="Male">
<label>Male</label>

<input type="radio" name="gender" value="Female">
<label>Female</label>
```

Checkboxes: Select multiple independent options.

```
None
<input type="checkbox" name="skills" value="HTML">
<label>HTML</label>

<input type="checkbox" name="skills" value="CSS">
<label>CSS</label>
```

Remember: Radio buttons in the same group share the same `name`.

4. Dropdown

```
None
<label for="course">Course:</label>

<select id="course" name="course">
    <option value="">Select course</option>
    <option value="CSE">CSE</option>
    <option value="ECE">ECE</option>
    <option value="IT">IT</option>
</select>
```

- `select` – Dropdown.
- `option` – Choice.
- `value` – Submitted option value.

5. Textarea

```
None
<textarea rows="4"
          cols="30"
```

```
          placeholder="Enter feedback">
</textarea>
```

Used for comments, feedback, address and descriptions.

6. Buttons

None

```
<button type="button">Click Me</button>
<button type="submit">Submit</button>
<button type="reset">Reset</button>
```

Type	Purpose
button	General button
submit	Submits form
reset	Resets form controls

7. HTML Tables

None

```
<table border="1">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Course</th>
        </tr>
    </thead>

    <tbody>
        <tr>
            <td>1</td>
            <td>Divya</td>
            <td>CSE</td>
        </tr>
    </tbody>
</table>
```

Tag	Purpose
table	Table
thead	Header group
tbody	Body group
tr	Row
th	Header cell
td	Data cell
tfoot	Footer group

colspan: Combines columns.

```
None
<td colspan="2">Combined columns</td>
```

rowspan: Combines rows.

```
None
<td rowspan="2">Combined rows</td>
```

8. Semantic HTML

Semantic elements describe the purpose of content.

```
None
<header>Website Header</header>

<nav>Navigation Links</nav>

<main>
    <section>
        <h2>About Us</h2>
    </section>

    <article>
        <h2>Latest News</h2>
```

```
    </article>
</main>

<aside>Related information</aside>

<footer>Copyright</footer>
```

Element	Purpose
header	Introductory content
nav	Navigation
main	Main content
section	Thematic grouping
article	Self-contained content
aside	Related content
footer	Footer information

Benefits: Better accessibility, meaningful structure and easier maintenance.

9. Linking CSS

Inside the HTML head:

```
None
<link rel="stylesheet" href="style.css">
```

Recommended project structure:

```
None
MyWebsite/
│
├── index.html
├── style.css
└── images/
    └── car.jpg
```

MODULE 3 – CSS FUNDAMENTALS

1. What is CSS?

CSS stands for Cascading Style Sheets.

CSS controls the appearance of HTML elements.

Examples:

- Colors
- Fonts
- Spacing
- Backgrounds
- Layouts
- Borders
- Responsive design

2. CSS Syntax

```
None
selector {
    property: value;
}
```

Example:

```
None
h1 {
    color: blue;
    font-size: 30px;
}
```

3. Three ways to apply CSS

Inline CSS:

```
None
<p style="color:red;">Hello</p>
```

Internal CSS:

```
None
<style>
    p {
        color: blue;
    }
</style>
```

External CSS:

HTML:

```
None
<link rel="stylesheet" href="style.css">
```

CSS:

```
None
p {
    color: green;
}
```

External CSS is generally preferred for maintainable projects.

4. CSS Selectors

Element selector:

```
None
p {
    color: blue;
}
```

Class selector:

```
None
.card {
    background: white;
}
```

ID selector:

```
None
#header {
    background: black;
}
```

Universal selector:

```
None
* {
    margin: 0;
    padding: 0;
}
```

Group selector:

```
None
h1, h2, p {
    font-family: Arial, sans-serif;
}
```

Descendant selector:

```
None
.card p {
    color: gray;
}
```

Child selector:

```
None
.card > p {
    color: red;
}
```

Pseudo-class:

```
None
button:hover {
    background: green;
```

```
}
```

Pseudo-element:

```
None
p::first-letter {
    font-size: 30px;
}
```

5. CSS Colors

```
None
h1 {
    color: red;
}

p {
    color: #ff0000;
}

div {
    background-color: rgb(255, 0, 0);
}
```

Common formats:

- Named colors.
- HEX.
- RGB.
- RGBA.
- HSL.

6. Fonts and Text

```
None
h1 {
    font-family: Arial, sans-serif;
    font-size: 32px;
    font-weight: bold;
    font-style: italic;
```

```
    text-align: center;
    text-decoration: underline;
    text-transform: uppercase;
    line-height: 1.5;
}
```

Property	Purpose
font-family	Font type
font-size	Text size
font-weight	Thickness
font-style	Italic or normal
text-align	Alignment
text-decoration	Underline etc.
text-transform	Uppercase/lowercase
line-height	Line spacing
letter-spacing	Character spacing

7. CSS Units

Unit	Meaning
px	Pixels
%	Relative percentage
em	Relative to font size
rem	Relative to root font size
vw	Viewport width
vh	Viewport height

Example:

```
None
.container {
```

```
    width: 80%;
    font-size: 1rem;
    min-height: 50vh;
}
```

MODULE 4 – CSS BOX MODEL AND POSITIONING

1. CSS Box Model

Every HTML element is represented by a rectangular box.

Four parts:

1. Content
2. Padding
3. Border
4. Margin

```None
.box {
    width: 200px;
    padding: 20px;
    border: 2px solid black;
    margin: 10px;
}
```

- Content: Actual content.
- Padding: Space inside border.
- Border: Boundary.
- Margin: Space outside border.

2. box-sizing

```None
* {
    box-sizing: border-box;
}
```

Value	Meaning
content-box	Width applies to content only
border-box	Width includes content, padding and border

Remember: `border-box` simplifies layout calculations.

3. Margin and Padding

```
None
.box {
    margin: 20px;
    padding: 15px;
}
```

Four-value shorthand:

```
None
.box {
    margin: 10px 20px 30px 40px;
}
```

Order: Top → Right → Bottom → Left.

4. Display Property

```
None
.block {
    display: block;
}

.inline {
    display: inline;
}

.inline-block {
    display: inline-block;
}

.hidden {
```

```
    display: none;
}
```

Value	Behavior
block	Starts on new line
inline	Flows within a line
inline-block	Inline flow with width/height support
none	Removed from layout

5. Position Property

None

```
.static {
    position: static;
}

.relative {
    position: relative;
}

.absolute {
    position: absolute;
}

.fixed {
    position: fixed;
}

.sticky {
    position: sticky;
    top: 0;
}
```

Position	Behavior
static	Normal document flow

relative	Offset from normal position
absolute	Positioned relative to containing block
fixed	Positioned relative to viewport
sticky	Sticks after scroll threshold

Overlay example:

```
None
.card {
    position: relative;
}

.overlay {
    position: absolute;
    bottom: 0;
    left: 0;
    width: 100%;
}
```

6. Overflow

```
None
.box {
    width: 200px;
    height: 100px;
    overflow: auto;
}
```

- visible
- hidden
- scroll
- auto

MODULE 5 – CSS FLEXBOX

1. What is Flexbox?

Flexbox is a one-dimensional layout system used to arrange items in rows or columns.

```
None
.container {
    display: flex;
}
```

2. flex-direction

```
None
.container {
    display: flex;
    flex-direction: row;
}
```

Values:

- row
- row-reverse
- column
- column-reverse

3. justify-content

Controls alignment along the main axis.

```
None
.container {
    display: flex;
    justify-content: center;
}
```

Common values:

- flex-start
- flex-end
- center
- space-between
- space-around
- space-evenly

4. align-items

Controls alignment along the cross axis.

```
None
.container {
    display: flex;
    align-items: center;
}
```

5. flex-wrap

```
None
.container {
    display: flex;
    flex-wrap: wrap;
}
```

Allows items to move onto another line.

6. gap

```
None
.container {
    display: flex;
    gap: 20px;
}
```

7. flex shorthand

```
None
.item {
    flex: 1;
}
```

Allows items to grow and share available space.

8. Centering elements

```
None
.container {
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
```

```
}
```

Remember: justify-content controls the main axis; align-items controls the cross axis.

MODULE 6 – CSS GRID

1. What is CSS Grid?

CSS Grid is a two-dimensional layout system for rows and columns.

```
None
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}
```

Creates three equal-width columns.

2. Important Grid properties

```
None
.container {
    display: grid;
    grid-template-columns: 1fr 2fr;
    grid-template-rows: auto auto;
    gap: 20px;
}
```

- grid-template-columns – Defines columns.
- grid-template-rows – Defines rows.
- gap – Space between items.
- grid-column – Controls column placement.
- grid-row – Controls row placement.

3. Responsive Grid

```
None
.container {
    display: grid;
    grid-template-columns:
        repeat(auto-fit, minmax(250px, 1fr));
    gap: 20px;
}
```

Automatically adapts columns to available space.

4. Flexbox vs Grid

Flexbox	Grid
One-dimensional	Two-dimensional
Row or column	Rows and columns
Navbar layouts	Full page layouts
Content-oriented	Layout-oriented

MODULE 7 – BACKGROUNDS, GRADIENTS AND EFFECTS

1. Background color

```
None
body {
    background-color: lightblue;
}
```

2. Background image

```
None
.hero {
    background-image: url("car.jpg");
    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
```

```
}
```

Property	Purpose
background-image	Sets image
background-size	Controls image sizing
background-position	Image positioning
background-repeat	Controls repetition

`cover` fills the container and may crop the image. `contain` fits the entire image.

3. Linear Gradient

```
None
.hero {
    background: linear-gradient(
        to right,
        #000000,
        #444444
    );
}
```

Multiple colors:

```
None
.hero {
    background: linear-gradient(
        135deg,
        #141e30,
        #243b55
    );
}
```

4. Transparent overlays

```
None
.overlay {
    background: rgba(0, 0, 0, 0.5);
```

```
}
```

The alpha value controls opacity:

- 0 = transparent.
- 1 = opaque.

5. Box Shadow

```
None
.card {
    box-shadow: 0 4px 12px rgba(0,0,0,0.2);
}
```

Syntax:

```
box-shadow: horizontal vertical blur spread color;
```

6. Border Radius

```
None
.card {
    border-radius: 12px;
}
```

7. Hover Effects

```
None
button:hover {
    background-color: blue;
    color: white;
}
```

8. Transitions

```
None
.card {
    transition: transform 0.3s ease;
}
```

```
.card:hover {
    transform: translateY(-5px);
}
```

9. Transform

```
None
.box:hover {
    transform: scale(1.05);
}
```

Common functions:

- scale()
- rotate()
- translate()
- skew()

10. Filter and Backdrop Filter

```
None
.background {
    filter: blur(5px);
}
```

```
None
.glass {
    background: rgba(255,255,255,0.2);
    backdrop-filter: blur(10px);
}
```

`filter` affects the element itself; `backdrop-filter` affects the area behind it.

MODULE 8 – RESPONSIVE WEB DESIGN

1. What is Responsive Design?

Responsive design makes webpages adapt to different screen sizes.

Examples:

- Desktop
- Laptop
- Tablet
- Mobile

2. Viewport Meta Tag

```
None
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

3. Media Queries

```
None
.container {
    display: flex;
}

@media (max-width: 768px) {
    .container {
        flex-direction: column;
    }
}
```

Applies the column layout when the viewport is 768px or narrower.

4. Responsive Images

```
None
img {
    max-width: 100%;
    height: auto;
}
```

5. Responsive Container

```
None
.container {
    width: 90%;
    max-width: 1200px;
```

```
    margin: auto;
}
```

6. Mobile-first design

```
None
.container {
    display: flex;
    flex-direction: column;
}

@media (min-width: 768px) {
    .container {
        flex-direction: row;
    }
}
```

Start with mobile styling, then add larger-screen rules.

7. Important responsive techniques

- Flexible widths.
- Relative units.
- Flexbox.
- Grid.
- Media queries.
- Responsive images.
- Appropriate maximum widths.

MODULE 9 – BOOTSTRAP FUNDAMENTALS

1. What is Bootstrap?

Bootstrap is a front-end CSS framework used to create responsive websites quickly.

It provides:

- Ready-made CSS classes.
- Responsive grid.
- Navbar.
- Cards.

- Buttons.
- Forms.
- Modals.
- Utility classes.

2. Adding Bootstrap using CDN

Inside HTML head:

```
None
<link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/boots
trap.min.css"
 rel="stylesheet">
```

Before closing body:

```
None
<script

src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstr
ap.bundle.min.js">
</script>
```

Note: The script above is Bootstrap's supplied component library, not a JavaScript learning topic. It is needed for interactive Bootstrap components such as collapsible navigation and modals. Check the official Bootstrap documentation for the current release.

3. Containers

```
None
<div class="container">
    Content
</div>
```

```
None
<div class="container-fluid">
    Full-width content
</div>
```

Class	Purpose
container	Responsive maximum widths
container-fluid	Full width at all breakpoints

4. Bootstrap Grid System

Bootstrap uses a 12-column grid.

```
None
<div class="container">
    <div class="row">
        <div class="col-6">Left</div>
        <div class="col-6">Right</div>
    </div>
</div>
```

Each column occupies 6 out of 12 columns.

5. Responsive Columns

```
None
<div class="row">
    <div class="col-12 col-md-6 col-lg-4">
        Card 1
    </div>

    <div class="col-12 col-md-6 col-lg-4">
        Card 2
    </div>

    <div class="col-12 col-md-6 col-lg-4">
        Card 3
    </div>
</div>
```

- Mobile: 12 columns.
- Medium: 6 columns.
- Large: 4 columns.

6. Bootstrap Breakpoints

Breakpoint	Minimum width
xs	Under 576px
sm	576px
md	768px
lg	992px
xl	1200px
xxl	1400px

7. Bootstrap Navbar

None

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">

        <a class="navbar-brand" href="#">
            My Website
        </a>

        <button class="navbar-toggler"
                type="button"
                data-bs-toggle="collapse"
                data-bs-target="#navbarMenu">

            <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse"
             id="navbarMenu">

            <ul class="navbar-nav ms-auto">
                <li class="nav-item">
                    <a class="nav-link" href="#home">
                        Home
                    </a>
                </li>

                <li class="nav-item">
                    <a class="nav-link" href="#about">
```

```
                        About
                    </a>
                </li>
            </ul>
        </div>
    </div>
</nav>
```

Important classes:

- navbar
- navbar-expand-lg
- navbar-dark
- bg-dark
- navbar-brand
- navbar-toggler
- navbar-nav
- nav-item
- nav-link
- collapse
- navbar-collapse
- ms-auto

8. Bootstrap Buttons

None

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Delete</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-outline-dark">Outline</button>
```

9. Bootstrap Cards

None

```
<div class="card" style="width: 18rem;">

    <img src="car.jpg"
         class="card-img-top"
         alt="Car">
```

```
    <div class="card-body">
        <h5 class="card-title">Luxury Car</h5>

        <p class="card-text">
            Comfortable rental car.
        </p>

        <a href="#" class="btn btn-primary">
            Book Now
        </a>
    </div>
</div>
```

Important classes:

- card
- card-img-top
- card-body
- card-title
- card-text

10. Bootstrap Forms

```
None
<form>
    <div class="mb-3">
        <label class="form-label">
            Email
        </label>

        <input type="email"
               class="form-control"
               placeholder="Enter email">
    </div>

    <button class="btn btn-primary" type="submit">
        Submit
    </button>
</form>
```

11. Bootstrap Spacing Utilities

Examples:

```
None
<div class="mt-3">Margin top</div>
<div class="mb-4">Margin bottom</div>
<div class="p-3">Padding</div>
<div class="px-4">Horizontal padding</div>
<div class="py-2">Vertical padding</div>
<div class="mx-auto">Horizontal auto margin</div>
```

Prefix	Meaning
m	Margin
p	Padding
t	Top
b	Bottom
s	Start
e	End
x	Horizontal
y	Vertical

12. Bootstrap Flex Utilities

```
None
<div class="d-flex justify-content-between align-items-center">
    <h2>Products</h2>

    <button class="btn btn-primary">
        Add
    </button>
</div>
```

Common classes:

Class	Purpose
d-flex	Flex container

flex-column	Column direction
flex-wrap	Wrapping
justify-content-center	Main-axis center
justify-content-between	Space between
align-items-center	Cross-axis center
gap-3	Gap

13. Background and Text Utilities

```
None
<div class="bg-dark text-white p-3">
    Dark section
</div>

<p class="text-primary">Primary text</p>
<p class="text-success">Success</p>
<p class="text-danger">Error</p>
```

14. Bootstrap Positioning

```
None
<nav class="navbar fixed-top bg-dark">
    ...
</nav>
```

Common classes:

- fixed-top
- fixed-bottom
- sticky-top

Fixed navigation is removed from normal document flow. Add suitable spacing so it does not cover content.

15. Smooth Scrolling

```
None
html {
    scroll-behavior: smooth;
```

```
}
```

```
None
<a href="#about">About</a>

<section id="about">
    About section
</section>
```

16. Bootstrap vs Custom CSS

Bootstrap	Custom CSS
Ready-made classes	Write your own styles
Faster development	More styling freedom
Standard components	Fully customized appearance
Framework conventions	Your own design rules

MODULE 10 – PRACTICAL WEBPAGE DEVELOPMENT

1. Recommended Folder Structure

```
None
MyWebsite/
│
├── index.html
├── css/
│   └── style.css
│
└── images/
    ├── logo.png
    └── car.jpg
```

2. Common Website Sections

1. Navbar.

2. Hero section.
3. About section.
4. Services/products section.
5. Cards.
6. Reviews.
7. Contact section.
8. Footer.

3. Car Rental Card Example

HTML:

```
None
<div class="car-card">
    <img src="car.jpg" alt="Luxury car">

    <h3>Luxury Sedan</h3>

    <p>Comfortable and spacious.</p>

    <p>₹3000 per day</p>

    <a href="#" class="book-btn">Book Now</a>
</div>
```

CSS:

```
None
.car-card {
    width: 300px;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.2);
    transition: transform 0.3s ease;
}

.car-card img {
    width: 100%;
    height: 180px;
    object-fit: cover;
}
```

```
.car-card:hover {
    transform: translateY(-5px);
}

.book-btn {
    display: inline-block;
    padding: 10px 20px;
    background: blue;
    color: white;
    text-decoration: none;
    border-radius: 5px;
}
```

4. Practical project ideas

- Personal portfolio website.
- Grocery store webpage.
- Student management information webpage.
- Car rental website.
- Electronics and gadgets shopping webpage.
- Restaurant website.
- Responsive landing page.

MODULE 11 – COMMON MISTAKES

1. Forgetting to link the CSS file.
2. Incorrect image paths.
3. Missing viewport meta tag.
4. Reusing the same ID.
5. Using fixed widths that break mobile layouts.
6. Forgetting to include Bootstrap CSS.
7. Forgetting Bootstrap's bundle when using interactive components.
8. Using absolute positioning without understanding the containing block.
9. Confusing margin and padding.
10. Using `justify-content` and `align-items` incorrectly.
11. Forgetting that fixed navbars can cover page content.
12. Using non-semantic elements unnecessarily.
13. Forgetting meaningful image alt text.
14. Ignoring mobile responsiveness.

MODULE 12 – QUICK REVISION SHEETS

HTML Quick Revision

Question	Answer
Full form?	HyperText Markup Language
Root element?	html
Visible content?	body
Link tag?	a
Image tag?	img
Form tag?	form
Table row?	tr
Header cell?	th
Unique identifier?	id
Reusable identifier?	class
Navigation element?	nav
Main content?	main
Line break?	br
Horizontal line?	hr

CSS Quick Revision

Question	Answer
Full form?	Cascading Style Sheets
Class selector?	.
ID selector?	#
Box model?	Content, padding, border, margin
Flex container?	display: flex
Grid container?	display: grid
Hover?	

Transition?	transition
Rounded corners?	border-radius
Shadow?	box-shadow
Responsive rules?	@media
Hide element?	display: none
Center flex items?	justify-content + align-items

Bootstrap Quick Revision

Question	Answer
What is Bootstrap?	Front-end CSS framework
Main container?	.container
Full-width container?	.container-fluid
Grid columns?	12
Row class?	.row
Column class?	.col-*
Button class?	.btn
Card class?	.card
Navbar class?	.navbar
Input class?	.form-control
Flex class?	.d-flex
Margin top?	.mt-*
Padding?	.p-*

MODULE 13 – INTERVIEW QUESTIONS

HTML

1. What is HTML?

HTML is a markup language used to structure webpages.

2. Difference between div and span?

div is a generic block-level container; span is a generic inline container.

3. What is semantic HTML?

Using elements that describe their content, such as header, nav, main and footer.

4. Difference between GET and POST?

GET retrieves resources and commonly sends parameters in the URL. POST sends data in the request body and is commonly used to submit or create data.

5. Difference between id and class?

ID is intended to identify one unique element; class can be reused.

CSS

6. Difference between margin and padding?

Margin is outside the border; padding is between content and border.

7. What is CSS specificity?

It determines the priority of competing CSS selectors within the cascade.

8. Difference between relative and absolute?

Relative offsets an element from its normal position. Absolute positions it relative to its containing block.

9. Difference between Flexbox and Grid?

Flexbox is one-dimensional; Grid is two-dimensional.

10. What is a media query?

A CSS rule that applies styles based on conditions such as viewport width.

11. What is box-sizing?

A CSS property that controls how element width and height are calculated.

12. What is z-index?

It controls stacking order for elements in the relevant stacking context.

Bootstrap

13. Why use Bootstrap?

It provides reusable components and responsive utilities.

14. What is the Bootstrap grid?

A responsive layout system based on 12 columns.

15. Difference between container and container-fluid?

Container uses responsive maximum widths; container-fluid uses full available width.

16. What is a breakpoint?

A viewport width at which responsive styles can change.

17. Can Bootstrap and custom CSS be combined?

Yes. Custom CSS can extend or override Bootstrap styling.

MODULE 14 – PRACTICAL REVISION CHECKLIST

HTML

- Create a complete HTML document.
- Practise headings and paragraphs.
- Create ordered and unordered lists.
- Add images and hyperlinks.
- Create registration forms.
- Practise labels, radio buttons and checkboxes.
- Create tables.
- Use semantic HTML.
- Link external CSS.

CSS

- Practise all three CSS methods.
- Understand selectors.
- Style fonts and colors.
- Understand the box model.
- Practise positioning.
- Create Flexbox layouts.
- Create Grid layouts.
- Create gradients and overlays.
- Practise hover effects and transitions.
- Create responsive webpages.

Bootstrap

- Add Bootstrap using CDN.
- Use containers and rows.
- Create responsive columns.
- Build a navbar.
- Create cards.
- Style forms and buttons.
- Practise spacing utilities.
- Practise Flexbox utilities.
- Build a responsive webpage.

FINAL REVISION METHOD

Understand → Write → Run → Observe → Practise → Revise

Do not memorize code blindly. Understand why every HTML element, CSS property and Bootstrap class is used.

This handbook covers HTML, CSS and Bootstrap only.